INTRODUCTION TO LINUX

What is an Operating System ?
Operating system is an interface between user and the computer hardware. The hardware of the computer cannot understand the human readable language as it works on binaries i.e. 0's and 1's. Also it is very tough for humans to understand the binary language, in such cases we need an interface which can translate human language to hardware and vice-versa for effective communication.

Types of Operating System:
Single User - Single Tasking Operating System
Single User - Multitasking Operating System
Multi User - Multitasking Operating System

Single User - Single Tasking Operating System
In this type of operating system only one user can log into the system and can perform only one task at a time.
E.g.: MS-DOS

Single User - Multi tasking operating System
This type of O/S supports only one user to log into the system but a user can perform multiple tasks at a time, browsing the internet while playing songs etc.
E.g.: Windows -98,Xp,vista,Seven etc.

Multi User - Multi Tasking Operating System
This type of O/S allows multiple users to log into the system and also each user can perform various tasks at a time. In a broader term multiple users can log in to the system and share the resources of the system at the same time.
E.g.: UNIX, LINUX etc.

HISTORY OF LINUX

The history of Linux is not just about one OS . it's a combination of ideas from UNIX, the GNU movement, and open-source philosophy. Let's go step by step in depth.

1. The Foundation: UNIX Era (1969–1980s)

- Linux roots begin with UNIX, developed in 1969 at Bell Labs.
- It was designed by Ken Thompson and Dennis Ritchie.
- UNIX was revolutionary because it supported multi-user and multitasking capabilities.
- It was written in the C language, making it portable across different machines.
- It introduced key concepts like file system hierarchy, "everything is a file," and shell scripting.
- Over time, UNIX became proprietary (paid and restricted), limiting access and innovation for developers.

2. The GNU Movement (1983)

- To overcome UNIX restrictions, Richard Stallman launched the GNU Project in 1983.
- The main goal of the GNU Project was to create a completely free and open UNIX-like operating system.
- GNU developed important components such as GCC (a compiler), Bash (a shell), and core utilities like `ls`, `cp`, and `mv`.
- However, GNU was incomplete because it did not have a kernel, which is essential for interacting with hardware.

3. Birth of Linux Kernel (1991)

- In 1991, Linus Torvalds, a student, started developing the Linux kernel.
- He was using MINIX, a small UNIX-like operating system, but found it limited and wanted a more powerful system.
- As a result, he began building his own kernel from scratch as a personal project.
- He announced his work on Usenet, saying it was “just a hobby,” which later became one of the most important technologies in the world.

4. First Linux Release (1991–1992)

- The first version of the Linux kernel (0.01) was released with very basic functionality and no graphical interface.
- Initially, it was simple and mainly used by developers and enthusiasts.
- Linus made a crucial decision to release Linux under the GNU General Public License.
- This allowed anyone to use, modify, and distribute Linux freely, making it truly open-source and powerful.

5. GNU + Linux = Complete OS

- The combination of GNU tools and the Linux kernel created a complete operating system called GNU/Linux.
- GNU provided essential utilities and software, while Linux acted as the core kernel.
- This combination solved the missing piece problem and made a fully functional OS available to users.

6. Rise of Linux Distributions (1993–2000)

- Different organizations started packaging Linux into complete systems known as distributions or distros.
- Early popular distributions included Slackware, Debian, and Red Hat Linux.
- These distributions made Linux easier to install, configure, and use for general users.
- They also introduced package management systems to simplify software installation and updates.

7. Enterprise & Server Growth (2000s)

- In the 2000s, Linux became widely adopted in enterprise environments and server infrastructure.
- Companies like IBM invested heavily in Linux development.
- Red Hat provided commercial support and enterprise-grade solutions.

- Businesses preferred Linux because it offered high stability, strong security, excellent performance, and no licensing costs.
- As a result, most web servers today run on Linux systems.

8. Linux in Mobile & Cloud (2010–Present)

- Linux expanded into mobile technology, with Android being built on the Linux kernel.
- It became the backbone of cloud computing platforms such as Amazon Web Services and Microsoft Azure.
- Linux is also essential in DevOps, where tools like Docker and Kubernetes are designed to run on Linux environments.

9. Linux Today

- Today, Linux is widely used in servers, cloud computing, supercomputers, embedded systems, and smartphones.
- It powers more than 90% of internet servers, making it a critical part of global infrastructure.
- A notable fact is that all of the world's top 500 supercomputers run on Linux.

Why Linux?

- Fresh implementation of UNIX APIs
- Open source development model
- Supports wide variety of hardware
- Supports many networking protocols and Configurations
- Fully supported

1) Linux is a UNIX-like OS: Linux is similar to UNIX as the various UNIX versions are to each other.

2) Multi-User and Multi-tasking: Linux is a multi-user and multi-tasking operating system. That means that more than one person can be logged on to the same Linux computer at the same time. The same user could even be logged into their account from two or more terminals at the same time; Linux is also Multi-Tasking. A user can have more than one program executing at the same time.

3) Wide hardware support: Red Hat Linux supports most pieces of modern x86 compatible PC hardware.

4) Fully Supported: Red Hat Linux is a fully supported distribution Red Hat Inc. provides many support programs for the smallest to the largest companies.

Key Advantages:

Security

- Strong permission system
- Less vulnerable to malware

Cost Effective

- Free and open-source
- No licensing cost

Customization

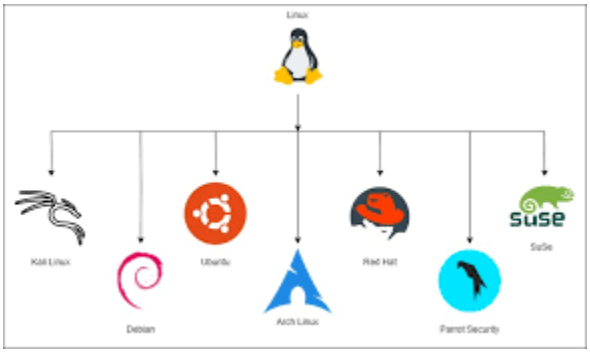
- Modify kernel, UI, tools

Performance

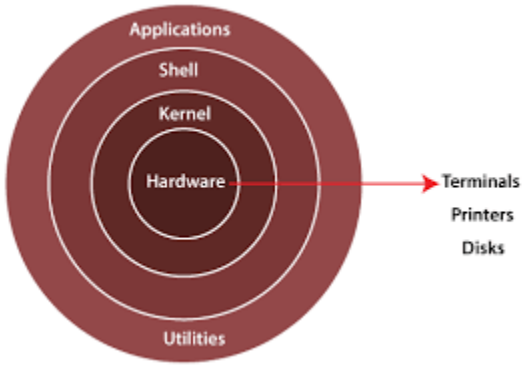
- Lightweight and efficient
- Best for servers & cloud

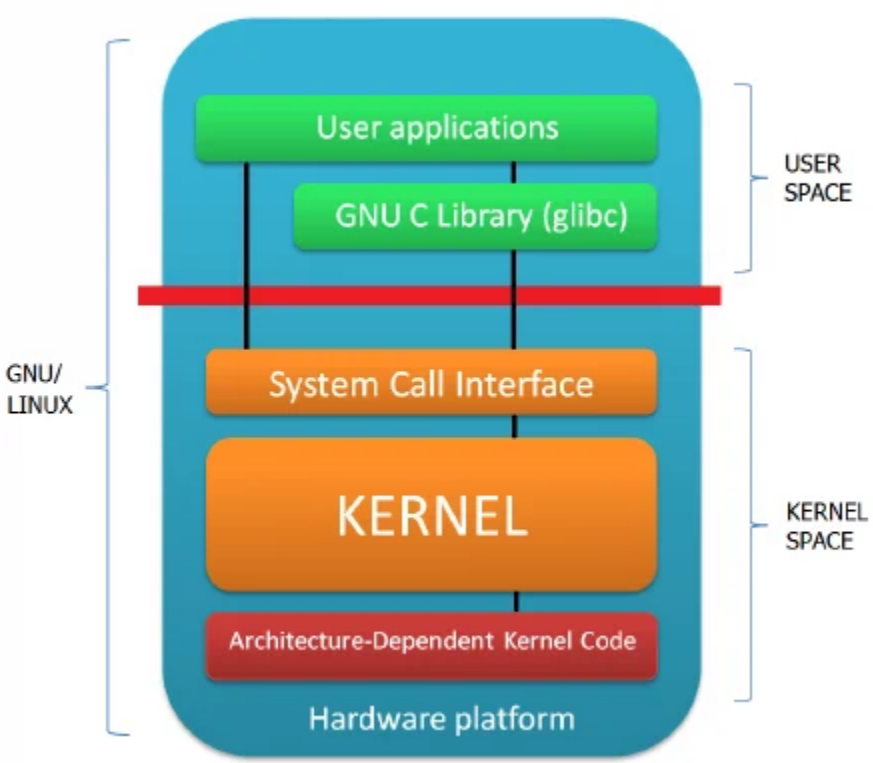
Widely Used

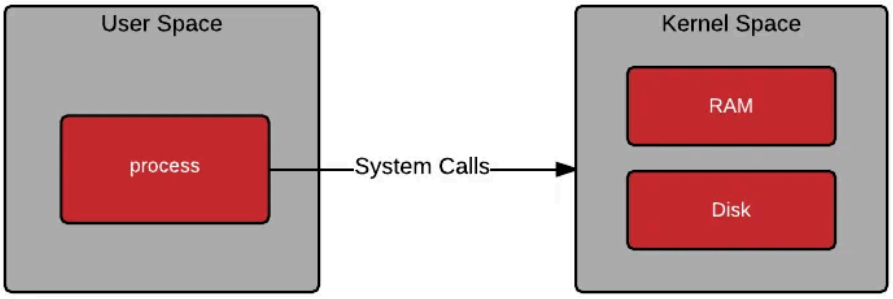
- Cloud (AWS, Azure, GCP)
- DevOps tools (Docker, Kubernetes)
- Servers, mobiles (Android)



Architecture of Linux







\

1. Hardware
Hardware consists of physical components like CPU, RAM, disk, and devices, which provide the basic resources required for the system to function.

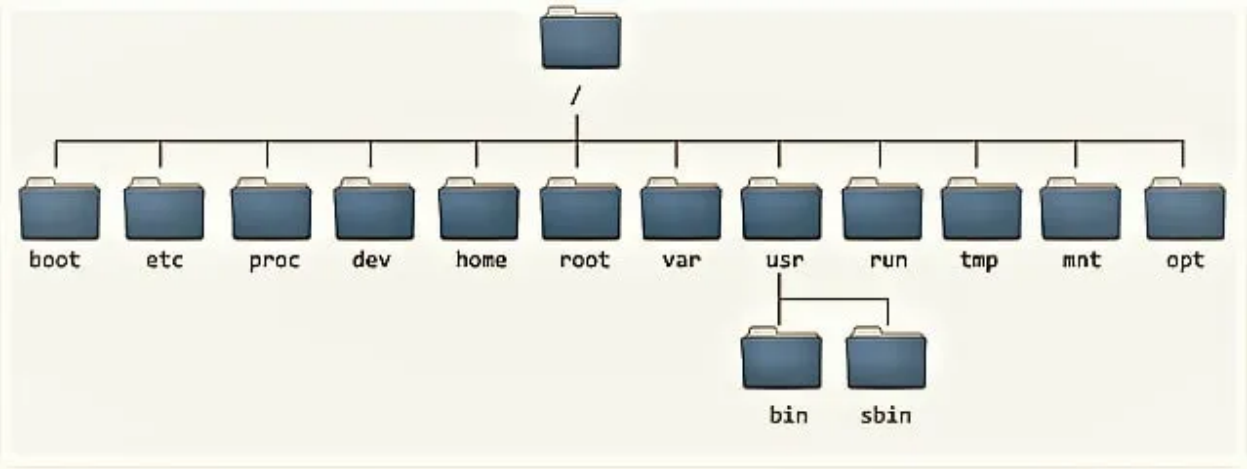
2. Kernel (Core)
The Linux kernel is the heart of the operating system that manages processes, memory, device drivers, and the file system, and it follows a monolithic design where all core functions run in a single space.

3. System Calls
System calls act as an interface between user applications and the kernel, allowing programs to request services like file operations using functions such as `open()`, `read()`, and `write()`.

4. Shell
The shell is a command interpreter that allows users to interact with the system by executing commands using interfaces like Bash or Zsh (e.g., `ls -l`).

5. User Space (Applications)
User space includes all applications like browsers, editors (vim, nano), and GUI tools that run above the kernel and interact with it through system calls.

Filesystem Hierarchy System (FHS)



Linux uses single rooted, inverted tree like file system hierarchy

---

/

- This is top level directory
- It is parent directory for all other directories
- It is called as ROOT directory
- It is represented by forward slash (/)
- C:\ of windows

/root

- it is home directory for root user (super user)
- It provides working environment for root user
- C:\Documents and Settings\Administrator

/home

- it is home directory for other users
- It provide working environment for other users (other than root)
- c:\Documents and Settings\username

/boot

- it contains bootable files for Linux Like vmlinuz (kernel). .... ntoskrnl
- Initrd (INITial Ram Disk)and
- GRUB (GRand Unified Boot loader). .... boot.ini, ntldr

/etc

- it contains all configuration files
- Like /etc/passwd..... User info
- /etc/resolv.conf... Preferred DNS
- /etc/dhcpd.conf .....DHCP server
- C:\windows\system32\dirvers\

/usr

- by default soft wares are installed in /usr directory
- (UNIX Sharable Resources)
- c:\program files

/opt

- It is optional directory for /usr
- It contains third party softwares
- c:\program files

/bin

- it contains commands used by all users
- (Binary files)

/sbin

- it contains commands used by only Super User (root)
- (Super user's binary files)

/dev

- it contains device files
- Like /dev/hda ... for hard disk
- /dev/cd rom ... for cd rom
- Similar to device manager of windows

/proc

- it contain process files
- Its contents are not permanent, they keep changing
- It is also called as Virtual Directory
- Its file contain useful information used by OS
- like /proc/meminfo ... information of RAM/SWAP
- /proc/cpuinfo ... information of CPU

/var

- it is containing variable data like mails, log files

/mnt

- it is default mount point for any partition
- It is empty by default

/media

- it contains all of removable media like CD-ROM, pen drive

/lib

- it contains library files which are used by OS
- It is similar to dll files of windows
- Library files in Linux are SO (shared object) files

/tmp

- The `/tmp` directory is used to store temporary files created by applications and users, and its contents are usually deleted automatically after a reboot.

/usr

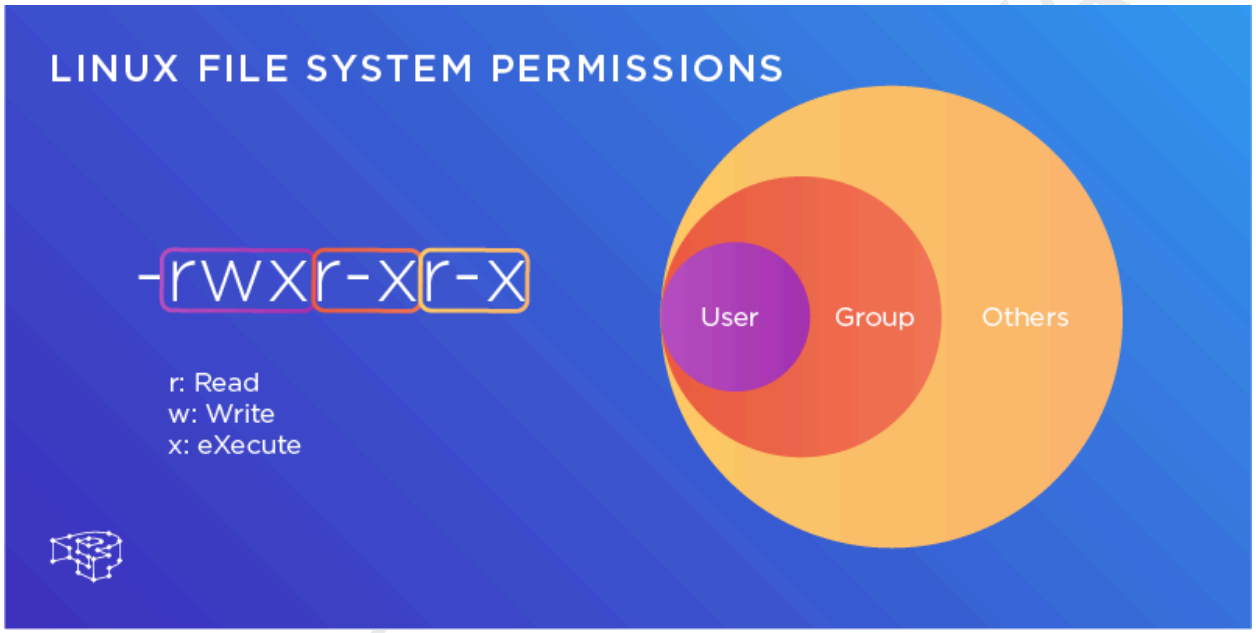
- User programs

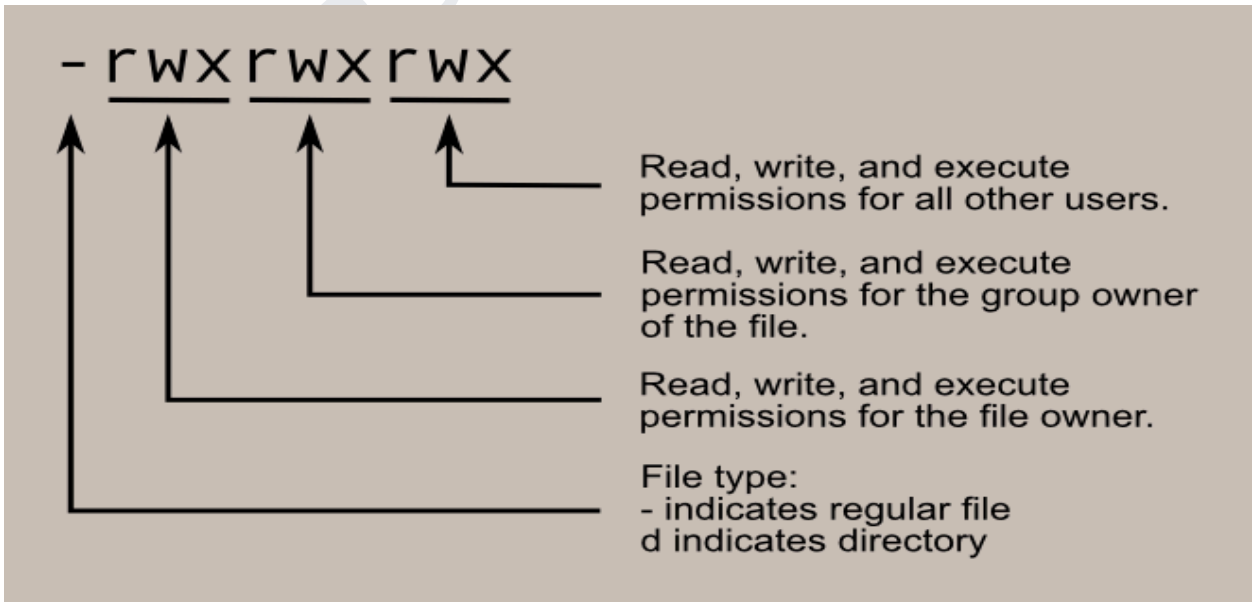
/run

- The `/run` directory stores temporary runtime data (like PID files and sockets) for active processes and is cleared on reboot.

Linux Permissions

- Linux permissions are a security mechanism that controls who can access a file or directory and what actions they can perform on it.
- Each file has three types of users: owner (user), group, and others, and each can have read (r), write (w), and execute (x) permissions.
- These permissions determine whether a user can view, modify, or run a file.





Every file/directory has 3 types of access levels:

- **User (u)** → Owner of the file
- **Group (g)** → Users in the same group
- **Others (o)** → Everyone else

-rwxr-xr-- 1 user group file.txt

`-` → file type

`rwx` → user permissions

`r-x` → group permissions

`r--` → others permissions

Permission Types

Each user type has 3 permissions:

- **r (read)** → View file content
- **w (write)** → Modify file
- **x (execute)** → Run file as a program

Combined:

rwx = read + write + execute

Each permission has a value:

- r = 4
- w = 2
- x = 1

User → rwx (7)

Group → r-x (5)

Others → r-x (5)

permission	Value
rwx	7
rw-	6
r-x	5
r–	4

Access modes are different on file and directory:

Permissions	Files	Directory
r	Open the file	‘Ls’ the contents of dir
w	Write, edit, append, delete file	Add/Del/Rename contents of dir
x	To run a command/shell script	To enter into dir using ‘cd’

Filetype+permission, links, owner, group name of owner, size in bytes, date of modification, file name

Permission can be set on any file/dir by two methods:-

1. Symbolic method (ugo)
2. Absolute methods (numbers)

1.Symbolic method (ugo):

- Symbolic mode: General form of symbolic mode is:
- # chmod [who] [+/-/=] [permissions] file
- Who → To whom the permissions to be assigned
- User/owner (u); group (g); others (o)

Example:

chmod 755 file.txt

Meaning:

- User → rwx (7)

- Group → r-x (5)
- Others → r-x (5)

Changing Permissions

Using `chmod`

- Symbolic:
➢ chmod u+x file.txt
➢ chmod g-w file.txt
➢ chmod o=r file.txt

- Numeric:
➢ chmod 644 file.txt
➢ chmod 777 file.txt

Ownership in Linux

Each file has:

- Owner (user)
- Group

Change ownership:

chown user file.txt
chown user:group file.txt

Access modes are different on file and directory:

Permissions	Files	Directory
r	Open the file	'Ls' the contents of dir
w	Write, edit, append, delete file	Add/Del/Rename contents of dir
x	To run a command/shell script	To enter into dir using 'cd'

Filetype+permission, links, owner, group name of owner, size in bytes, date of modification, file name

File vs Directory Permissions

📄 File:

- r → read content
- w → edit file
- x → execute

📁 Directory:

- r → list files
- w → create/delete files
- x → enter directory
- Without `x`, you **cannot access the directory**, even if `r` is present.

Special Permissions

SUID (Set User ID)

- File runs with owner's permissions

chmod 4755 file

SGID (Set Group ID)

- Inherit group permissions

chmod 2755 dir

Sticky Bit

- Only owner can delete files

chmod 1755 /tmp

Example:

- `/tmp` uses sticky bit

Default Permissions

Default permissions are controlled by `umask`

Example:

umask 022

- Files → 644
- Directories → 755

Linux Storage Management: Partitions, File Systems & Logical Volume Management (LVM)

Managing Partitions & File Systems

- Disk partitioning in Linux is the process of dividing a physical disk into smaller sections called partitions to organize and manage data efficiently.
- Each partition can be formatted with a file system such as ext4, xfs, or btrfs, which defines how data is stored and retrieved.
- Tools like `fdisk`, `parted`, and `lsblk` are used to create, view, and manage disk partitions.
- After creating a partition, it must be formatted using commands like `mkfs` before it can be used.
- Partitions are mounted to directories using the `mount` command so that users can access their contents.
- The `/etc/fstab` file is used to configure automatic mounting of partitions during system boot.

- Proper partition management helps in improving performance, organizing data, and isolating system areas.

File Systems

- A file system is a method used by Linux to store, organize, and manage files on a disk.
- Common Linux file systems include ext4 (most widely used), xfs (high performance), and btrfs (advanced features).
- The file system controls how data is written, read, and indexed on storage devices.
- Commands like `df`, `du`, and `mount` help in monitoring and managing file systems.

- File systems also support permissions and ownership, which ensure secure access to data.
- Journaling file systems (like ext4 and xfs) help in recovering data in case of system crashes.

Logical Volume Management

- Logical Volume Management (LVM) is a method of managing disk storage that provides flexibility compared to traditional partitions.
- It allows combining multiple physical disks into a single volume group, making storage easier to manage.
- LVM consists of three main components: Physical Volumes (PVs), Volume Groups (VGs), and Logical Volumes (LVs).
- Physical Volumes are actual disks or partitions, which are grouped into a Volume Group.
- Logical Volumes are created from the Volume Group and act like flexible partitions that can be resized.
- LVM allows resizing of volumes (increase or decrease) without downtime, which is very useful in real-time environments.
- Snapshots in LVM allow taking backups of volumes without interrupting system operations.
- LVM is widely used in servers and cloud environments because of its scalability and flexibility.

USER AND GROUP MANAGEMENT

User administration

- In Linux/Unix, a user is a person or account that uses the system.
- There can be one or multiple users on a Linux system at the same time.
- Each user is identified by a username and a user ID (UID).
- The username is a user-friendly name (like a person's name) used for login and identification.
- The UID is a numeric value used internally by the operating system to identify the user.
- Although username and UID are related, they are different and should not be confused or used interchangeably.
- Each user must have a unique UID, meaning no two users should share the same user ID.
- If two users share the same UID, they will have the same permissions and file ownership, making them appear as the same user to the system.
- This can create security and management issues, so it is not allowed.

- The `useradd` command ensures that each user has a unique UID and prevents duplication.

Some Important Points related to Users:

- Users and groups are used to control access to files and resources
- Users login to the system by supplying their username and password
- Every file on the system is owned by a user and associated with a group
- Every process has an owner and group affiliation, and can only access the resources its owner or group can access.
- Every user of the system is assigned a unique user ID number (the UID)
- Users name and UID are stored in /etc/passwd
- User's password is stored in /etc/shadow in encrypted form.
- Users are assigned a home directory and a program that is run when they login (Usually a shell) Users cannot read, write or execute each other's files without permission.

Types of users In Linux and their attributes:

TYPE	EXAMPLE	USER ID(UID)	GROUP ID(GID)	HOME DIRECTORY	SHELL
Super user	Root	0	0	/root	/bin/bash
System user	ftp, ssh, apache nobody	1 to 499	1 to 499	/var/ftp , etc	/sbin/nologin
Normal user	Visitor, ktuser,etc	500 to 60000	500 to 60000	/home/user name	/bin/bash

In Linux there are three types of users.

1. Super user or root user

- Super user or the root user is the most powerful user.
- He is the administrator user.

2. System user

- System users are the users created by the softwares or applications.
- For example if we install Apache it will create a user apache.
- These kinds of users are known as system users.

3. Normal user

- Normal users are the users created by root user.
- They are normal users like Rahul, Musab etc.
- Only the root user has the permission to create or remove a user.

Whenever a user is created in Linux things created by default:-

- A home directory is created(/home/username)
- A mail box is created(/var/spool/mail)
- unique UID & GID are given to user

Linux uses UPG (User Private Group) scheme

It means that whenever a user is created is has its own private group
For Example if a user is created with the name Rahul, then a primary group for that user will be Rahul only

There are two important files a user administrator should be aware of.

1. "/etc/passwd"
2. "/etc/shadow"

Each of the above mentioned files have specific formats.

1. **/etc/passwd**

```
[root@ktlinux ~]# head /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
```

The above fields are

- root =name
- x= link to password file i.e. /etc/shadow
- 0 or 1= UID (user id)
- 0 or 1=GID (group id)
- root or bin = comment (brief information about the user)
- /root or /bin = home directory of the user
- /bin/bash or /sbin/nologin = shell Page 97 of 274

2. /etc/shadow

The fields are as follows,

- root = User name
- :$1fdsfsgsdfsdkffefje = Encrypted password
- 14757 = Days since that password was last changed.
- 0 = Days after which password must be changed.
- 99999 = Days before password is to expire that user is warned.
- 7 = Days after the password is expires that the user is disabled.
- A reserved field.

Password Complexity Requirements:

- A root user can change password of self and of any user in the system, there are no rules for root to assign a password. Root can assign any length of password either long or short, it can be alphabet or numeric or both. On the whole there is no limitation for root for assigning a password.
- A normal user can change only its password. Valid password for a normal user should adhere to the following rules
- It should be at least 7 characters but not more than 255 characters.
- At least one character should be Upper case
- At least one character should be Lower case
- At least one character should be a symbol, and one character should be a number.
- It should not match the previous password.
- It cannot have a sequence (ex: 123456 or abcdef)
- The login name and the password cannot be same

Note: For security reasons don't keep the password based on date of birth because it can easily be hacked.

Group administration

- Users are assigned to groups with unique group ID numbers (the GID)
- The group name and GID are stored in /etc/group
- Each user is given their own private group
- They can also be added to their groups to gain additional access
- All users in a group can share files that belong to the group

Each user is a member of at least one group, called a primary group. In addition, a user can be a member of an unlimited number of secondary groups. Group membership can be used to control the files that a user can read and edit. For example, if two users are working on the same project you might put them in the same group so they can edit a particular file that other users cannot access.

- A user’s primary group is defined in the /etc/passwd file and Secondary groups are defined in the /etc/group file.
- The primary group is important because files created by this user will inherit that group affiliation.